Practice 10 — Build Once, Run Anywhere

Objective: package a small web app into a container image, run several containers from that one image, keep the data outside the containers, and then run the same image "in the cloud": from **Amazon ECR** on an EC2 instance (Track A) or from a local registry behind a load balancer (Track B). These are the skills from today's lecture.

Timebox: in class: ~25 min for Part 1, ~30 min for Part 2 (Track B is faster, ~20 min). If you've been stuck on one step for more than 10 minutes, stop and ask. Don't push through alone.

> **How you're checked:** this practice is done in class and checked **orally**. There is no report. When you finish, keep your terminal (and console, on Track A) open. The instructor will ask you to show the items in the *Definition of done* and to explain what you did.

What you need

- **Docker** (Docker Desktop, OrbStack, or Docker Engine on Linux) and a bash shell (Linux, macOS, or WSL / Git Bash on Windows). Check with `docker version` and `docker compose version`.
- Part 2 Track A also needs AWS Academy Learner Lab and the AWS CLI.

Pick your track for Part 2

Part 1 is the same for everyone. For Part 2, pick **one** track:

	Track A — real AWS	**Track B — no AWS**
Needs	AWS Academy Learner Lab (or your own account)	Docker only
Registry	Amazon ECR	`registry:2` container on `localhost:5001`
Where the image runs	One EC2 instance (Docker installed by user data)	3 containers behind HAProxy
Slow step	Instance boot + Docker install: 2–4 min	none

The ideas are the same on both tracks:

AWS (Track A)	Track B equivalent
Amazon ECR repository	`registry:2` container on `localhost:5001`
`aws ecr get-login-password | docker login`	Not needed: Docker trusts a registry on `localhost`
Image tag `<account>.dkr.ecr.us-east-1.amazonaws.com/narxoz-web:v1`	`localhost:5001/narxoz-web:v1`
EC2 instance profile (IAM role) pulls the image	Your Docker pulls from the local registry
User data: install Docker, `docker run`	`compose.yaml` service `web` with `image: localhost:5001/...`
ALB + Auto Scaling group (week 9, stretch)	HAProxy + `--scale web=3`

Definition of done

Be ready to **show** each of these:

- □ Your `Dockerfile` and the image you built (`docker images`).
- □ **Two containers from the same image** answering with different colors and different hostnames.
- □ A rebuild after a code change where `pip install` shows **`CACHED`**.
- □ Data in PostgreSQL that **survived** **`docker compose down`** (not `down -v`).
- □ Part 2:
  - Track A: the image in **ECR**, and `curl` to the EC2 instance answering from your container. **No access keys** on the instance.
  - Track B: the image in the local registry (`/v2/_catalog`), and requests through HAProxy answered by **3 different containers**.
- □ Cleanup done and shown.

Part 1 — Docker on your laptop (everyone)

Work in a new folder:

```
mkdir lab10 && cd lab10
```

P1. Run someone else's image

```
docker run -d --name web -p 8080:80 nginx:1.27
docker ps
curl -s localhost:8080 | head -n 4        # the nginx welcome page
docker logs web                           # one line per request
docker exec -it web sh                    # a shell INSIDE the container
#   cat /etc/os-release ; hostname ; exit
docker stop web && docker rm web
```

`-p 8080:80` means "host port 8080 → container port 80". The `hostname` you saw inside is the container ID.

P2. The app

Create `app.py`:

```
import os
import socket

from flask import Flask

app = Flask(__name__)
COLOR = os.environ.get("APP_COLOR", "blue")


@app.get("/")
def hello():
    return f"Hello from {socket.gethostname()} | color={COLOR} | v1\n"


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

Create `requirements.txt` with one line:

```
flask==3.1.3
```

Create `.dockerignore`:

```
__pycache__/
*.pyc
.git
.env
```

P3. Write the Dockerfile

Create `Dockerfile` and replace each `TODO`:

```
FROM python:3.12-slim
WORKDIR /app
# TODO 1: copy ONLY requirements.txt, then install it with
#         pip install --no-cache-dir -r requirements.txt
RUN useradd --create-home appuser
# TODO 2: copy app.py
ENV APP_COLOR=blue
EXPOSE 8000
# TODO 3: switch to the user "appuser", then start the app with
#         CMD ["python", "app.py"]
```

Hints: `COPY <file> .` copies into `/app`. `RUN` runs at build time, `CMD` when the container starts. `USER appuser` comes before `CMD`.

Build and check:

```
docker build -t narxoz-web:v1 .
docker images narxoz-web
docker history narxoz-web:v1          # one line per layer
```

P4. One image, two containers

```
docker run -d --name blue  -p 8001:8000 narxoz-web:v1
docker run -d --name green -p 8002:8000 -e APP_COLOR=green narxoz-web:v1
curl localhost:8001          # Hello from 4a528be586a0 | color=blue | v1
curl localhost:8002          # Hello from bf65d706f072 | color=green | v1
docker exec blue whoami      # appuser, not root
```

Two different hostnames (container IDs), two colors — and you built only **one** image. The color came from the environment, not from the image.

P5. The build cache

Change `v1` to `v2` in the `return` line of `app.py`, then rebuild with the build log visible:

```
docker build --progress=plain -t narxoz-web:v2 . 2>&1 | grep -E 'pip install|CACHED'
```

`RUN pip install ...` is reported as **`CACHED`**: only `COPY app.py` and the steps after it ran again.

Now see what happens with the wrong order. Create `Dockerfile.slow`:

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
RUN useradd --create-home appuser
USER appuser
CMD ["python", "app.py"]
```

```
docker build -f Dockerfile.slow -t narxoz-web:slow .     # first build
# change v2 to v3 in app.py, then:
docker build --progress=plain -f Dockerfile.slow -t narxoz-web:slow . 2>&1 \
  | grep -E 'pip install|CACHED'
```

This time `pip install` runs again: `COPY . .` changed, so every step after it is rebuilt. Change `app.py` back to `v1` when you're done.

```
docker rm -f blue green
```

P6. Compose: app + database, data outside the containers

Create `.env` (this file is in `.dockerignore` and must never go into git):

```
POSTGRES_PASSWORD=change-me-lab10
```

Create `compose.yaml`:

```
services:
  web:
    build: .
    ports:
      - "8080:8000"
    environment:
      APP_COLOR: orange
    depends_on:
      - db

  db:
    image: postgres:17
    env_file: .env
    volumes:
      - dbdata:/var/lib/postgresql/data

volumes:
  dbdata:
```

The database publishes **no port**: only `web` is reachable from your laptop. Start it and write some data:

```
docker compose up -d --build
until docker compose exec -T db pg_isready -U postgres -q; do sleep 1; done
curl localhost:8080                       # color=orange
docker compose exec -T db psql -U postgres \
  -c "CREATE TABLE notes(id serial PRIMARY KEY, txt text);" \
  -c "INSERT INTO notes(txt) VALUES ('survives down');"
```

Now remove the containers, but not the volume:

```
docker compose down                       # containers and network gone
docker volume ls | grep dbdata            # the volume is still there
docker compose up -d
until docker compose exec -T db pg_isready -U postgres -q; do sleep 1; done
docker compose exec -T db psql -U postgres -c "SELECT txt FROM notes;"
#  survives down
```

The database container is brand new, but the data was in the **volume**. Leave the stack running until you've shown it; it's removed in the cleanup.

Part 2, Track A — Amazon ECR and EC2 (AWS Academy Learner Lab)

> **Using AWS Academy Learner Lab**

1. Open the **AWS Academy Learner Lab** course → **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**. Wait until the dot next to **AWS** turns green (1–3 min).
2. Use region **`us-east-1`**. Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed.
3. You need Docker **and** the AWS CLI on the same machine, so use your laptop: copy the credentials from **AWS Details** → **AWS CLI** → **Show** into `~/.aws/credentials`. They change every session.
4. Key pair: use the lab's **`vockey`** (**AWS Details** → **Download PEM** saves `labsuser.pem`). You only need it if you want to SSH in.
5. Per the Learner Lab Readme, the lab provides an instance profile called **`LabInstanceProfile`** (role `LabRole`). We use it so the instance can pull from ECR without keys. If ECR or the instance profile is refused (`AccessDenied`, `UnauthorizedOperation`), do **Track B** instead.
6. When you're done, clean up (A5) and click **End Lab**.

A1. Create the repository and log in

```
export AWS_REGION=us-east-1
aws ecr create-repository --repository-name narxoz-web \
  --image-scanning-configuration scanOnPush=true

ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
REG=$ACCOUNT.dkr.ecr.us-east-1.amazonaws.com
aws ecr get-login-password | docker login --username AWS --password-stdin $REG
```

`Login Succeeded` means your Docker can now push to your ECR.

A2. Build for the instance's CPU, tag and push

EC2 `t3` instances are **amd64**. On an Apple Silicon Mac your normal build is arm64 and would fail on the instance with `exec format error`, so build for the target explicitly (it's fine on Intel/AMD laptops too):

```
docker buildx build --platform linux/amd64 -t narxoz-web:v1-amd64 --load .
docker image inspect narxoz-web:v1-amd64 --format '{{.Architecture}}'   # amd64

docker tag narxoz-web:v1-amd64 $REG/narxoz-web:v1
docker push $REG/narxoz-web:v1

aws ecr describe-images --repository-name narxoz-web \
  --query 'imageDetails[].[imageTags[0],imageSizeInBytes]' --output table
aws ecr describe-image-scan-findings --repository-name narxoz-web \
  --image-id imageTag=v1 --query 'imageScanFindings.findingSeverityCounts'
```

The scan can take a minute; if it says the scan isn't complete, run it again later.

A3. A security group and the user data

```
VPC=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true \
  --query 'Vpcs[0].VpcId' --output text)
SG=$(aws ec2 create-security-group --group-name lab10-web-sg \
  --description "lab10 web" --vpc-id $VPC --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
```

Create `user-data.sh`. It writes your registry address in, so run this `cat` exactly as shown (with `$REG` set from A1):

```
cat > user-data.sh <<EOF
#!/bin/bash
dnf install -y docker
systemctl enable --now docker
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin $REG
docker run -d --restart unless-stopped -p 80:8000 \
  -e APP_COLOR=green $REG/narxoz-web:v1
EOF
cat user-data.sh          # check that the registry address is filled in
```

There are **no access keys** in this script. On the instance, `aws ecr get-login-password` uses the instance profile's role.

A4. Launch the instance and test

```
IID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.micro --key-name vockey \
  --security-group-ids $SG \
  --iam-instance-profile Name=LabInstanceProfile \
  --user-data file://user-data.sh \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab10-web}]' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-running --instance-ids $IID
IP=$(aws ec2 describe-instances --instance-ids $IID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
echo $IP

sleep 120                 # Docker install + image pull
curl http://$IP           # Hello from <container id> | color=green | v1
```

If it isn't answering yet, wait another minute. Still nothing after 5 minutes? Connect with **EC2 Instance Connect** in the console and check `sudo cat /var/log/cloud-init-output.log` and `sudo docker ps -a`.

Stretch (optional):

- Put the same user data into a **launch template** and run two instances behind an ALB, exactly like week 9 — every server now runs the same image.
- Or describe how an **ECS service on Fargate** would replace the instance: task definition with the ECR image and port 8000, desired count 2, an ALB target group of type `ip`.

A5. Cleanup (required)

```
aws ec2 terminate-instances --instance-ids $IID
aws ec2 wait instance-terminated --instance-ids $IID
aws ec2 delete-security-group --group-id $SG
aws ecr delete-repository --repository-name narxoz-web --force
```

Then run the **Part 1 cleanup** below and click **End Lab**.

Part 2, Track B — a local registry and 3 replicas (no AWS)

Port **5001**, not 5000: on macOS, port 5000 is used by AirPlay Receiver.

B1. Run a registry and push your image

```
docker run -d --name registry -p 5001:5000 registry:2

docker tag narxoz-web:v1 localhost:5001/narxoz-web:v1
docker push localhost:5001/narxoz-web:v1

curl -s localhost:5001/v2/_catalog                 # {"repositories":["narxoz-web"]}
curl -s localhost:5001/v2/narxoz-web/tags/list     # {"name":"narxoz-web","tags":["v1"]}
```

Prove it's really pulled from the registry, the way a fresh EC2 instance would get it:

```
docker image rm localhost:5001/narxoz-web:v1
docker pull localhost:5001/narxoz-web:v1
```

B2. Three replicas behind HAProxy (the week 9 setup)

In a new folder:

```
mkdir ../lab10b && cd ../lab10b
```

Save as `haproxy.cfg`:

```
resolvers docker
  nameserver dns1 127.0.0.11:53
  resolve_retries 3
  timeout resolve 1s
  timeout retry   1s
  hold valid      5s

defaults
  mode http
  timeout connect 3s
  timeout client  10s
  timeout server  10s

frontend http_in
  bind :80
  default_backend web_servers

backend web_servers
  balance roundrobin
  option httpchk GET /
  default-server inter 2s fall 2 rise 2
  server-template web 10 web:8000 check resolvers docker init-addr none

frontend stats
  bind :8404
  stats enable
  stats uri /
  stats refresh 5s
```

Save as `compose.yaml`. The `web` service **pulls** the image from your registry instead of building it — like the EC2 instance pulling from ECR:

```
services:
  web:
    image: localhost:5001/narxoz-web:v1
    environment:
      APP_COLOR: purple
    restart: always
    deploy:
      replicas: 3

  lb:
    image: haproxy:3.0-alpine
    volumes:
      - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
    ports:
      - "8090:80"
      - "8404:8404"
    depends_on:
      - web
```

```
docker compose up -d
sleep 8
for i in $(seq 1 6); do curl -s localhost:8090; done
```

Three different hostnames take turns. Stop one: the site keeps answering.

```
docker stop lab10b-web-2
sleep 5
for i in $(seq 1 6); do curl -s -o /dev/null -w '%{http_code} ' localhost:8090; done; echo
docker compose up -d --scale web=3 --no-recreate     # back to 3
```

Stats page: http://localhost:8404.

B3. Cleanup (Track B)

```
docker compose down
cd ../lab10
docker rm -f registry
```

Then run the Part 1 cleanup below.

Part 1 cleanup (everyone, at the end)

```
cd lab10                                    # if you're not there already
docker compose down -v                      # NOW the volume is deleted too
docker image rm narxoz-web:v1 narxoz-web:v2 narxoz-web:slow lab10-web
docker image rm narxoz-web:v1-amd64 localhost:5001/narxoz-web:v1 2>/dev/null
cd .. && rm -rf lab10 lab10b
```

Be ready to explain

1. What's the difference between a container and a virtual machine? Why does a container start so much faster?
2. Image vs container: what happens to each when you run `docker rm`?
3. Why is `requirements.txt` copied and installed **before** `app.py`? What did you see in P5?
4. Where did your PostgreSQL data live, and why did it survive `docker compose down` but not `down -v`?
5. Why should containers be stateless? Where does state go on AWS?
6. Track A: how did the EC2 instance get permission to pull from ECR without access keys? Why did you build with `--platform linux/amd64`? Track B: what played the role of ECR, and of the ALB?
7. How would an **ECS service on Fargate** replace your instance (or your 3 replicas)? What are a task definition and a service?
8. Why should a password never be in the Dockerfile or the image?

Common mistakes

- **`Bind for 0.0.0.0:8080 failed: port is already allocated`.** Another container (or app) uses that port. `docker ps` to find it, or pick another host port.
- **`curl` gets "connection refused" but the container is running.** The app listens on `127.0.0.1` inside the container. It must listen on `0.0.0.0` (our `app.run(host="0.0.0.0", ...)`).
- **The container exits immediately.** `docker logs <name>` shows why — usually a typo in `CMD` or a missing file.
- **`pip install` is never `CACHED`.** You copied all files (`COPY . .`) before installing, or changed `requirements.txt`.
- **The data is gone after restart.** You ran `docker compose down -v`, or the volume line is missing in `compose.yaml`.
- **`no basic auth credentials` when pushing (Track A).** The ECR login expired or was never done. Re-run `aws ecr get-login-password | docker login ...` (the token lasts 12 hours).
- **`exec format error` on the instance (Track A).** The image was built for arm64. Rebuild with `--platform linux/amd64` and push again.
- **The instance doesn't answer after 5 minutes (Track A).** Check `/var/log/cloud-init-output.log`: if `docker login` failed, the instance profile is missing or the registry address in `user-data.sh` is empty (`$REG` wasn't set when you created the file).

- **`http: server gave HTTP response to HTTPS client` (Track B).** You used your computer's IP instead of `localhost`. Docker only trusts a plain-HTTP registry on `localhost`.
- **Containers aren't called `lab10b-web-2` (Track B).** Compose names containers after the folder. Use `docker compose ps` to see the real names.